WOXSEN UNIVERSITY

DatabaseManagementSystems—CourseProject Report

Design and Implementation of a Database Management System for

Dairy Farm Herd and Milk Collection Management System

Submitted by	Surakanti Nymisha Reddy
Roll number	25WU0102279
Course	Database Management Systems (DBMS)
Faculty	Dr. Kiran Mayee Adavala
Academic year	2026 – 2027
Project type	Individual project

Contents

2. Abstract
3. Introduction and Problem Statement
4. Objectives and Scope
5. Software and Hardware Requirements
6. ER Diagram
7. Relational Schema and Normalization
8. Data Dictionary
9. SQL Commands Used (DDL and DML)
10. Queries with Outputs
11. UI Design and Screenshots
12. Implementation Details
13. Testing
14. Conclusion and Future Enhancements
15. References
16. Appendix: GitHub Repository Link

Section 1 is the cover page.

2. Abstract

Dairy cooperatives and milk-collection centres gather milk twice a day from many small farmers. Each collection has to be measured for quantity, fat percentage and SNF (solids-not-fat) percentage, priced according to a rate chart, and eventually paid out to the farmer. In many centres this information still lives in paper registers or unconnected spreadsheets, which leads to lost records, slow payment calculation and no reliable health history for the animals.

This project designs and implements a relational database, **dairy_farm_db**, for a dairy farm herd and milk collection system. The database has six normalised tables (Farmer, Cattle, Milk Collection, Rate Chart, Payment and Health Record) linked by primary and foreign keys. A web interface built with HTML, CSS and JavaScript, served by a Node.js and Express backend, connects to MySQL and lets a user view, insert and delete records in every table. The amount due to each farmer is calculated entirely in SQL by joining collection entries to the rate band that matches the fat percentage.

The report covers the ER model, the relational schema and its normalization to Third Normal Form, the data dictionary, the DDL and DML commands, sample queries with their outputs, the user interface, implementation details and a test plan.

Keywords: DBMS, MySQL, ER diagram, normalization, dairy management, milk collection, Node.js, Express.

3. Introduction and Problem Statement

3.1 Background

India is the largest milk producer in the world, and most of that milk comes from small and marginal farmers who each keep a handful of animals. Village-level collection centres and dairy cooperatives gather milk every morning and evening, test it, and pay the farmers periodically. The price paid for a litre of milk is not fixed: it depends on the fat and SNF content found in the sample, so every single delivery has to be recorded with its quantity and quality readings.

Alongside milk collection, a farm or cooperative has to keep track of the animals themselves: which animal belongs to which farmer, its breed and age, whether it is currently lactating, and when it was last vaccinated or treated by a veterinarian.

3.2 Problem Statement

Records for herds, daily milk collection and farmer payments are commonly kept on paper or in separate spreadsheets. This causes the following problems:

- Duplicate, inconsistent or lost farmer and cattle records.
- Slow and error-prone calculation of payments, because each delivery must be matched by hand to the correct fat/SNF price band.
- No quick way to find which animals are overdue for vaccination, since health history is scattered.
- Reports such as daily yield, top suppliers or amount due have to be compiled manually.

The problem is therefore to design a database that stores farmers, cattle, milk collections, price bands, payments and health records in a structured and consistent way, and to build an interface through which authorised staff can record and review this data.

3.3 Why a Relational Database

Thedataisnaturallyrelational. One farmer owns many animals, supplies milk many times and receives many payments; one animal has many health records. These one-to-many relationships are exactly what primary and foreign keys enforce. A DBMS also provides constraints that keep the data valid, transactions that keep it consistent, and SQL for producing reports directly from the stored data.

4. Objectives and Scope

4.1 Objectives

- Design an ER model for dairy farm herd management and milk collection.
- Convert the ER model into a relational schema in Third Normal Form with proper keys and constraints.
- Implement the schema in MySQL and populate it with realistic sample data.
- Calculate the amount due to each farmer in SQL from quantity and fat-based rate bands.
- Track animal health history and identify animals overdue for vaccination.
- Build a web user interface connected to MySQL that supports insertion, deletion and viewing of records.
- Test the system with defined test cases, including constraint violations.

4.2 Scope

Inscope: farmer registry; cattle registry linked to owners; daily milk collection by shift with fat and SNF readings; configurable rate chart; payment records per period; veterinary health records; a web UI for insert, delete and view on all six tables; a report that computes amount due for any date.

Out of scope (future work): user login and roles, record editing from the UI, automatic generation of payment statements, integration with milk-testing hardware, SMS notifications, and mobile applications.

5. Software and Hardware Requirements

5.1 Software Requirements

Component	Requirement
Operating system	Windows 10 / 11 (also runs on macOS and Linux)
Database server	MySQL 8.0 or later
Database tool	MySQL Workbench 8.0
Runtime	Node.js 18 or later with npm
Backend libraries	express, mysql2, cors, dotenv
Frontend	HTML5, CSS3, JavaScript (ES2020), no framework
Code editor	Visual Studio Code
Browser	Google Chrome, Microsoft Edge or Firefox (current version)
Version control	Git and GitHub

5.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2 GHz	Intel Core i5 / Ryzen 5 or better
Memory (RAM)	4 GB	8 GB
Storage	2 GB free	5 GB free
Display	1280 × 720	1920 × 1080
Network	Not needed to run; internet needed once for npm install	Broadband

6. ER Diagram

Theconceptualdesign has six entities. The diagram below uses rectangles for entities, diamonds for relationships and 1 / N labels for cardinality.

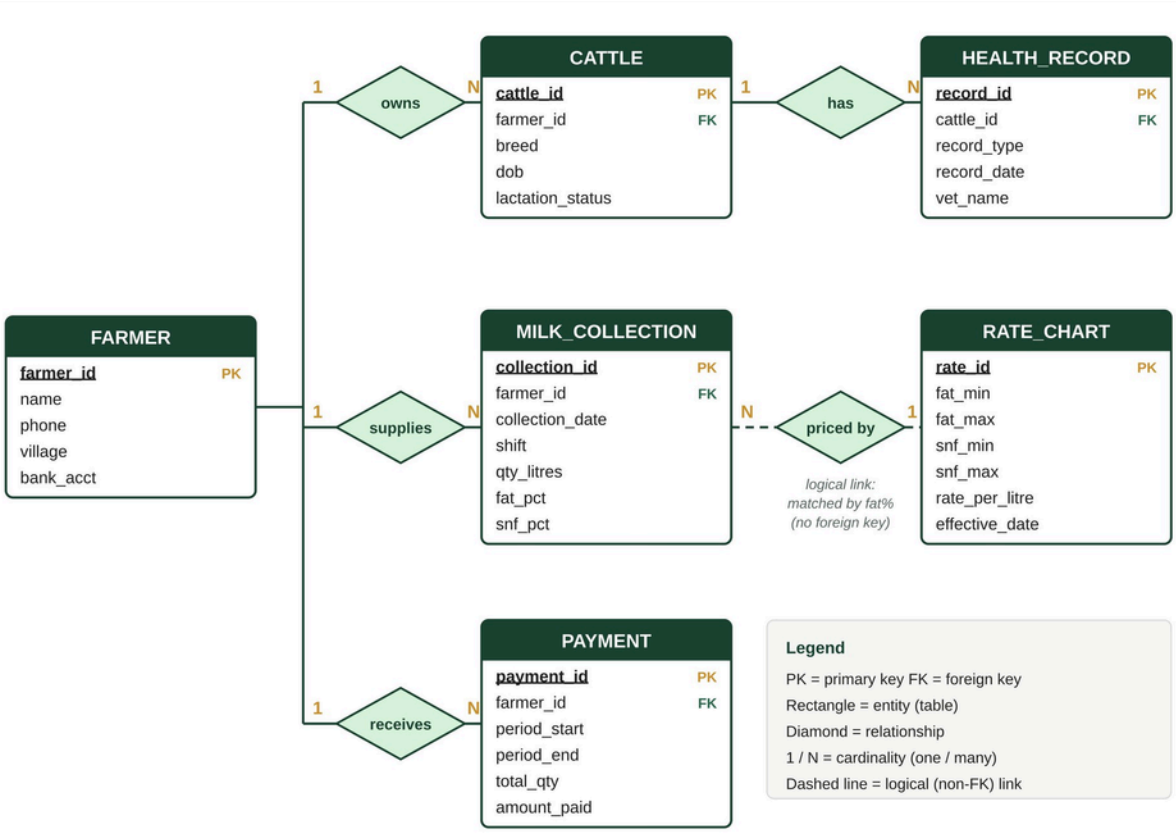


Figure 6.1: Entity–Relationship diagram of the Dairy Farm Herd and Milk Collection system

6.1 Entities

Entity	Description	Primary key
FARMER	A registered milk supplier who owns cattle	farmer_id
CATTLE	An individual animal belonging to one farmer	cattle_id
MILK_COLLECTION	One delivery: a farmer's milk in one shift on one date, with	collection_id
RATE_CHART	fat and SNF readings	rate_id
PAYMENT	A price band: rate per litre for a fat and SNF range	payment_id
HEALTH_RECORD	A payment made to a farmer for a period	record_id
	A vaccination, checkup or treatment for one animal	

6.2 Relationships and Cardinality

Relationship	Entities	Cardinality	Implemented by
owns	FARMER – CATTLE	1 : N	cattle.farmer_id (FK)
supplies	FARMER – MILK_COLLECTION	1 : N	milk_collection.farmer_id (FK)
receives	FARMER – PAYMENT	1 : N	payment.farmer_id (FK)
has	CATTLE – HEALTH_RECORD	1 : N	health_record.cattle_id (FK)
priced by	MILK_COLLECTION – RATE_CHART	N : 1	Range join on fat% (no FK)

The relationship between a milk collection and the rate chart is deliberately not a foreign key. A rate band is chosen by comparing the measured fat percentage with the band's range, not by storing a band identifier in every collection row. This keeps collection entries independent of price changes: when new rates are introduced, past collections stay untouched.

7. Relational Schema and Normalization

7.1 Relational Schema

Primarykeysareunderlinedin the ER diagram and marked PK below; foreign keys are marked FK.

```
FARMER(farmer_id PK, name, phone UNIQUE, village, bank_acct)
CATTLE(cattle_id PK, farmer_id FK -> FARMER, breed, dob, lactation_status)
RATE_CHART(rate_id PK, fat_min, fat_max, snf_min, snf_max,
           rate_per_litre, effective_date)
MILK_COLLECTION(collection_id PK, farmer_id FK -> FARMER, collection_date,
                shift, qty_litres, fat_pct, snf_pct)
PAYMENT(payment_id PK, farmer_id FK -> FARMER, period_start, period_end,
        total_qty, amount_paid)
HEALTH_RECORD(record_id PK, cattle_id FK -> CATTLE, record_type,
              record_date, vet_name)
```

Referential actions

Foreign key	References	ON DELETE	ON UPDATE
cattle.farmer_id	farmer.farmer_id	RESTRICT	CASCADE
milk_collection.farmer_id	farmer.farmer_id	RESTRICT	CASCADE
payment.farmer_id	farmer.farmer_id	RESTRICT	CASCADE
health_record.cattle_id	cattle.cattle_id	CASCADE	CASCADE

RESTRICT prevents deleting a farmer who still has cattle, milk collections or payments, so financial and herd history cannot be lost by accident. CASCADE on health records means that when an animal is removed, its own health history is removed with it.

7.2 Functional Dependencies

- farmer_id → name, phone, village, bank_acct; and phone → farmer_id (phone is a candidate key).
- cattle_id → farmer_id, breed, dob, lactation_status.
- collection_id → farmer_id, collection_date, shift, qty_litres, fat_pct, snf_pct.
- rate_id → fat_min, fat_max, snf_min, snf_max, rate_per_litre, effective_date.
- payment_id → farmer_id, period_start, period_end, total_qty, amount_paid.
- record_id → cattle_id, record_type, record_date, vet_name.

7.3 Normalization

Unnormalised form. If everything were kept in one sheet, each row would repeat the farmer's name, phone and village, the animal's breed, and the price per litre on every delivery. Changing a farmer's phone number would mean editing many rows, and deleting the last delivery of a farmer would erase the farmer's details.

First Normal Form (1NF). Every attribute holds a single atomic value and there are no repeating groups. A farmer's several animals are stored as separate rows of CATTLE instead of a comma- separated list, and repeated deliveries are separate rows of MILK_COLLECTION. Every table has a primary key.

Second Normal Form (2NF). All tables use a single-column primary key, so partial dependency on part of a composite key cannot occur. Every non-key attribute depends on the whole key.

Third Normal Form (3NF). There are no transitive dependencies. Farmer details are stored only in FARMER and referred to by farmer_id elsewhere. Prices are stored only in RATE_CHART and are not copied into each collection row. Animal details live only in CATTLE. In every table, non-key attributes depend only on the key.

BCNF. For every functional dependency listed above, the determinant is a candidate key (farmer_id, phone, cattle_id, collection_id, rate_id, payment_id, record_id), so the schema also satisfies Boyce–Codd Normal Form.

7.4 Design Decisions

- **Stored payment totals.** payment.total_qty and payment.amount_paid can be calculated from collections, but they are stored deliberately. A payment is a settled historical fact; if the rate chart changes next month, the amount already paid must not change.

- **Non-overlapping rate bands.** Bands are stored as 3.50–3.99, 4.00–4.49 and 4.50–4.99. If one band ended at 4.00 and the next began at 4.00, a fat value of exactly 4.0 would match both bands in a BETWEEN join, the row would be counted twice, and the amount due would be overstated.
- **Surrogate keys.** Integer AUTO_INCREMENT keys keep joins fast and avoid using changeable data (such as a phone number) as an identifier.

8. Data Dictionary

Each table below lists column names, MySQL data types, constraints and a short description.

8.1farmer

Column	Data type	Constraints	Description
farmer_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique farmer identifier
name	VARCHAR(100)	NOT NULL	Farmer's full name
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number
village	VARCHAR(100)	NOT NULL	Village of residence
bank_acct	VARCHAR(30)	NULL allowed	Bank account reference for payment

8.2 cattle

Column	Data type	Constraints	Description
cattle_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique animal identifier
farmer_id	INT	NOT NULL, FOREIGN KEY → farmer	Owner of the animal
breed	VARCHAR(50)	NOT NULL	Breed, e.g. Jersey, Holstein
dob	DATE	NULL allowed	Date of birth
lactation_status	ENUM('Lactating','Dry')	DEFAULT 'Dry'	Current milking status

8.3 rate_chart

Column	Data type	Constraints	Description
rate_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique rate band identifier
fat_min	DECIMAL(4,2)	NOT NULL	Lower fat % of band (inclusive)
fat_max	DECIMAL(4,2)	NOT NULL	Upper fat % of band (inclusive)
snf_min	DECIMAL(4,2)	NOT NULL	Lower SNF % of band
snf_max	DECIMAL(4,2)	NOT NULL	Upper SNF % of band
rate_per_litre	DECIMAL(6,2)	NOT NULL	Price paid per litre in rupees
effective_date	DATE	NOT NULL	Date from which the rate applies

8.4 milk_collection

Column	Data type	Constraints	Description
collection_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique collection entry
farmer_id	INT	NOT NULL, FOREIGN KEY → farmer	Supplying farmer
collection_date	DATE	NOT NULL	Date of collection
shift	ENUM('Morning','Evening')	NOT NULL	Collection shift
qty_litres	DECIMAL(6,2)	NOT NULL	Milk quantity in litres
fat_pct	DECIMAL(4,2)	NOT NULL	Measured fat percentage
snf_pct	DECIMAL(4,2)	NOT NULL	Measured SNF percentage

8.5 payment

Column	Data type	Constraints	Description
payment_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique payment identifier
farmer_id	INT	NOT NULL, FOREIGN KEY → farmer	Farmer who is paid
period_start	DATE	NOT NULL	First day covered
period_end	DATE	NOT NULL	Last day covered
total_qty	DECIMAL(8,2)	NOT NULL	Total litres in the period
amount_paid	DECIMAL(10,2)	NOT NULL	Amount paid in rupees

8.6 health_record

Column	Data type	Constraints	Description
record_id	INT	PRIMARYKEY, AUTO_INCREMENT	Unique record identifier
cattle_id	INT	NOT NULL, FOREIGN KEY → cattle	Animal concerned
record_type	ENUM('VACCINATION','CHECKUP','TREATMENT')	NOT NULL	Kind of veterinary event
record_date	DATE	NOT NULL	Date of the event
vet_name	VARCHAR(100)	NULL allowed	Attending veterinarian

Indexes. idx_milk_farmer_date on milk_collection(farmer_id, collection_date), idx_cattle_farmer on cattle(farmer_id) and idx_health_cattle on health_record(cattle_id) speed up the joins and date filters used by the queries.

9. SQL Commands Used (DDL and DML)

9.1 Data Definition Language (DDL)

Create the database and select it:

```
DROP DATABASE IF EXISTS dairy_farm_db;
CREATE DATABASE dairy_farm_db;
USE dairy_farm_db;
```

Create the tables (parent tables first so foreign keys can refer to them):

```
CREATE TABLE farmer (
    farmer_id      INT PRIMARY KEY AUTO_INCREMENT,
    name           VARCHAR(100) NOT NULL,
    phone          VARCHAR(15) NOT NULL UNIQUE,
    village        VARCHAR(100) NOT NULL,
    bank_acct      VARCHAR(30)
);

CREATE TABLE cattle (
    cattle_id          INT PRIMARY KEY AUTO_INCREMENT,
    farmer_id          INT NOT NULL,
    breed              VARCHAR(50) NOT NULL,
    dob                DATE,
    lactation_status   ENUM('Lactating', 'Dry') DEFAULT 'Dry',
    FOREIGN KEY (farmer_id) REFERENCES farmer(farmer_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE rate_chart (
    rate_id  fat_min  INT          PRIMARY          KEY
    fat_max           AUTO_INCREMENT,
    snf_min           DECIMAL(4,2)      NOT      NULL,
    snf_max           DECIMAL(4,2)      NOT      NULL,
    rate_per_litre    DECIMAL(4,2)      NOT      NULL,
    effective_date    DECIMAL(4,2)      NOT      NULL,
                      DECIMAL(6,2) NOT NULL,
);                    DATE NOT NULL

CREATE TABLE milk_collection (
    collection_id     INT PRIMARY KEY AUTO_INCREMENT,
    farmer_id         INT NOT NULL,
    collection_date DATE NOT NULL,
    shift             ENUM('Morning',     'Evening')     NOT
    qty_litres        NULL, DECIMAL(6,2) NOT NULL,
    fat_pct           DECIMAL(4,2) NOT NULL,
    snf_pct           DECIMAL(4,2) NOT NULL,
    FOREIGN KEY (farmer_id) REFERENCES farmer(farmer_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE payment (
    payment_id     INT          PRIMARY          KEY
    farmer_id      AUTO_INCREMENT,    INT    NOT
    period_start   NULL,
    period_end     DATE NOT NULL,
    total_qty      DATE NOT NULL,
                   DECIMAL(8,2) NOT NULL,
```

```
    amount_paid    DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (farmer_id) REFERENCES farmer(farmer_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE health_record (
    record_id       INT PRIMARY KEY AUTO_INCREMENT,
    cattle_id       INT NOT NULL,
    record_type     ENUM('VACCINATION', 'CHECKUP', 'TREATMENT') NOT NULL,
    record_date     DATE NOT NULL,
    vet_name        VARCHAR(100),
    FOREIGN KEY (cattle_id) REFERENCES cattle(cattle_id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE INDEX idx_milk_farmer_date ON milk_collection(farmer_id, collection_date);
CREATE INDEX idx_cattle_farmer    ON cattle(farmer_id);
CREATE INDEX idx_health_cattle    ON health_record(cattle_id);
```

Confirm that the tables exist:

```
SHOW TABLES;
```

Tables_in_dairy_farm_db
cattle
farmer
health_record
milk_collection
payment
rate_chart

Output of SHOW TABLES;

9.2 Data Manipulation Language (DML)

Sample data is inserted with INSERT statements, parent tables first:

```
INSERT INTO farmer (farmer_id, name, phone, village, bank_acct) VALUES
(1, 'Ramesh Kumar', '9876543210', 'Anantapur', 'SBIN0001234-55671'),
(2, 'Suresh Reddy', '9123456780', 'Kadapa',          'HDFC0004567-
(3, 'Lakshmi Devi', '9988776655', 'Kurnool',         88932'), 'ICIC0009876-
                                                     22110');
INSERT INTO cattle (cattle_id, farmer_id, breed, dob, lactation_status) VALUES
(101, 1, 'Jersey',          '2021-03-15', 'Lactating'),
(102, 1, 'Holstein', '2020-07-22', 'Dry'),
(103, 2, 'Sahiwal',       '2019-11-05', 'Lactating');

INSERT INTO rate_chart (rate_id, fat_min, fat_max, snf_min, snf_max,
                        rate_per_litre, effective_date) VALUES
(1, 3.50, 3.99, 8.00, 8.49, 32.00, '2026-01-01'),
(2, 4.00, 4.49, 8.50, 8.99, 35.00, '2026-01-01'),
(3, 4.50, 4.99, 9.00, 9.49, 38.00, '2026-01-01');

INSERT INTO milk_collection (collection_id, farmer_id, collection_date, shift,
                             qty_litres, fat_pct, snf_pct) VALUES
(5001, 1, '2026-09-20', 'Morning', 12.5, 4.2, 8.6),
```

```
(5002, 1, '2026-09-20', 'Evening', 10.0, 4.0, 8.4),
(5003, 2, '2026-09-20', 'Morning',         8.3, 3.8, 8.2);

INSERT INTO payment (payment_id, farmer_id, period_start, period_end,
                             total_qty, amount_paid) VALUES
(1, 1, '2026-08-01', '2026-08-31', 620.0, 19800.00),
(2, 2, '2026-08-01', '2026-08-31', 245.0, 7900.00);

INSERT INTO health_record (record_id, cattle_id, record_type,
                                    record_date, vet_name) VALUES
(1,  101,  'VACCINATION',  '2026-03-01',  'Dr.  Naidu'),  (2,
102, 'VACCINATION', '2025-12-15', 'Dr. Naidu'),
(3, 103, 'CHECKUP',          '2026-08-10', 'Dr. Rao');
```

9.3 SELECT Statements and Sample Outputs

Thescreenshots belowshowtheresultof SELECT* onevery table, run in the MySQL command-line client against the database dairy_farm_db. The same results are tabulated afterwards for readability.

```
mysql> USE dairy_farm_db;
Database changed
mysql>
mysql> SELECT * FROM farmer;
+-----------+--------------+------------+-----------+-------------------+
| farmer_id | name         | phone      | village   | bank_acct         |
+-----------+--------------+------------+-----------+-------------------+
|         1 | Ramesh Kumar | 9876543210 | Anantapur | SBIN0001234-55671 |
|         2 | Suresh Reddy | 9123456780 | Kadapa    | HDFC0004567-88932 |
|         3 | Lakshmi Devi | 9988776655 | Kurnool   | ICIC0009876-22110 |
+-----------+--------------+------------+-----------+-------------------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM cattle;
+-----------+-----------+----------+------------+------------------+
| cattle_id | farmer_id | breed    | dob        | lactation_status |
+-----------+-----------+----------+------------+------------------+
|       101 |         1 | Jersey   | 2021-03-15 | Lactating        |
|       102 |         1 | Holstein | 2020-07-22 | Dry              |
|       103 |         2 | Sahiwal  | 2019-11-05 | Lactating        |
+-----------+-----------+----------+------------+------------------+
3 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM rate_chart;
+---------+---------+---------+---------+---------+----------------+----------------+
| rate_id | fat_min | fat_max | snf_min | snf_max | rate_per_litre | effective_date |
+---------+---------+---------+---------+---------+----------------+----------------+
|       1 |    3.50 |    3.99 |    8.00 |    8.49 |          32.00 | 2026-01-01     |
|       2 |    4.00 |    4.49 |    8.50 |    8.99 |          35.00 | 2026-01-01     |
|       3 |    4.50 |    4.99 |    9.00 |    9.49 |          38.00 | 2026-01-01     |
+---------+---------+---------+---------+---------+----------------+----------------+
3 rows in set (0.009 sec)
```

*Figure 9.1: MySQL output of SELECT * FROM farmer, cattle and rate_chart*

```
mysql>
mysql> SELECT * FROM milk_collection;
+---------------+-----------+-----------------+---------+------------+---------+---------+
| collection_id | farmer_id | collection_date | shift   | qty_litres | fat_pct | snf_pct |
+---------------+-----------+-----------------+---------+------------+---------+---------+
|          5001 |         1 | 2026-09-20      | Morning |      12.50 |    4.20 |    8.60 |
|          5002 |         1 | 2026-09-20      | Evening |      10.00 |    4.00 |    8.40 |
|          5003 |         2 | 2026-09-20      | Morning |       8.30 |    3.80 |    8.20 |
+---------------+-----------+-----------------+---------+------------+---------+---------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM payment;
+------------+-----------+--------------+------------+-----------+-------------+
| payment_id | farmer_id | period_start | period_end | total_qty | amount_paid |
+------------+-----------+--------------+------------+-----------+-------------+
|          1 |         1 | 2026-08-01   | 2026-08-31 |    620.00 |    19800.00 |
|          2 |         2 | 2026-08-01   | 2026-08-31 |    245.00 |     7900.00 |
+------------+-----------+--------------+------------+-----------+-------------+
2 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM health_record;
+-----------+-----------+-------------+-------------+-----------+
| record_id | cattle_id | record_type | record_date | vet_name  |
+-----------+-----------+-------------+-------------+-----------+
|         1 |       101 | VACCINATION | 2026-03-01  | Dr. Naidu |
|         2 |       102 | VACCINATION | 2025-12-15  | Dr. Naidu |
|         3 |       103 | CHECKUP     | 2026-08-10  | Dr. Rao   |
+-----------+-----------+-------------+-------------+-----------+
3 rows in set (0.008 sec)
```

*Figure 9.2: MySQL output of SELECT * FROM milk_collection, payment and health_record*

```
SELECT * FROM farmer;
```

farmer_id	name	phone	village	bank_acct
1	Ramesh Kumar	9876543210	Anantapur	SBIN0001234-55671
2	Suresh Reddy	9123456780	Kadapa	HDFC0004567-88932
3	Lakshmi Devi	9988776655	Kurnool	ICIC0009876-22110

*Output of SELECT * FROM farmer;*

```
SELECT * FROM cattle;
```

cattle_id	farmer_id	breed	dob	lactation_status
101	1	Jersey	2021-03-15	Lactating
102	1	Holstein	2020-07-22	Dry
103	2	Sahiwal	2019-11-05	Lactating

*Output of SELECT * FROM cattle;*

```
SELECT * FROM rate_chart;
```

rate_id	fat_min	fat_max	snf_min	snf_max	rate_per_litre	effective_date
1	3.50	3.99	8.00	8.49	32.00	2026-01-01
2	4.00	4.49	8.50	8.99	35.00	2026-01-01
3	4.50	4.99	9.00	9.49	38.00	2026-01-01

*Output of SELECT * FROM rate_chart;*

```
SELECT * FROM milk_collection;
```

collection_id	farmer_id	collection_date	shift	qty_litres	fat_pct	snf_pct
5001	1	2026-09-20	Morning	12.50	4.20	8.60
5002	1	2026-09-20	Evening	10.00	4.00	8.40
5003	2	2026-09-20	Morning	8.30	3.80	8.20

*Output of SELECT * FROM milk_collection;*

```
SELECT * FROM payment;
```

payment_id	farmer_id	period_start	period_end	total_qty	amount_paid
1	1	2026-08-01	2026-08-31	620.00	19800.00
2	2	2026-08-01	2026-08-31	245.00	7900.00

*Output of SELECT * FROM payment;*

```
SELECT * FROM health_record;
```

record_id	cattle_id	record_type	record_date	vet_name
1	101	VACCINATION	2026-03-01	Dr. Naidu
2	102	VACCINATION	2025-12-15	Dr. Naidu
3	103	CHECKUP	2026-08-10	Dr. Rao

*Output of SELECT * FROM health_record;*

10. Queries with Outputs

10.1 Query Presented in Presentation-II: Amount Due per Farmer

Purpose. Calculatehow much each farmer should be paid for the milk collected on 20 September 2026. Every collection row is joined to the rate band whose fat range contains the measured fat percentage, the litres are multiplied by that band's rate, and the results are summed per farmer.

```
SELECT
    f.name,
    SUM(m.qty_litres * r.rate_per_litre) AS amount_due
FROM farmer f
JOIN milk_collection m ON m.farmer_id = f.farmer_id
JOIN rate_chart r ON m.fat_pct BETWEEN r.fat_min AND r.fat_max
WHERE m.collection_date = '2026-09-20'
GROUP BY f.name;
```

name	amount_due
Ramesh Kumar	787.5000
Suresh Reddy	265.6000

Output of Query 10.1 (MySQL shows four decimal places because DECIMAL(6,2) × DECIMAL(6,2) has scale 4; wrap the SUM in ROUND(..., 2) for 787.50 and 265.60)

Working.

Farmer	Collection	Qty (L)	Fat %	Matched band	Rate (₹/L)	Amount (₹)
Ramesh Kumar	5001 Morning	12.5	4.2	4.00 – 4.49 4.00	35.00	437.50
Ramesh Kumar	5002 Evening	10.0	4.0	– 4.49 3.50 –	35.00	350.00
Suresh Reddy	5003 Morning	8.3	3.8	3.99	32.00	265.60

Ramesh Kumar: 437.50 + 350.00 = **₹787.50**. Suresh Reddy: **₹265.60**. Because the bands do not overlap, the entry with fat exactly 4.0 matches only one band and is counted once.

10.2 Cattle Overdue for Vaccination

Purpose. Listevery animal thathasno vaccination record in the last 180 days, or none at all. A LEFT JOIN keeps animals with no vaccination, which then show NULL.

```
SELECT c.cattle_id,   c.breed,   h.record_dateASlast_vaccination
FROM cattle c
LEFT JOIN health_record h
         ON h.cattle_id = c.cattle_id
        AND h.record_type = 'VACCINATION'
WHERE h.record_date IS NULL
    OR h.record_date < DATE_SUB(CURRENT_DATE, INTERVAL 180 DAY);
```

cattle_id	breed	last_vaccination
101	Jersey	2026-03-01 2025-
102	Holstein	12-15
103	Sahiwal	NULL

Output of Query 10.2 when run on 13 October 2026 (result depends on CURRENT_DATE)

10.3 Daily Milk Yield per Farmer

Purpose. Totallitresperfarmerperdaywith the average fat and SNF.

```
SELECT f.name,m.collection_date,
       SUM(m.qty_litres)          AS total_litres,
       ROUND(AVG(m.fat_pct), 2) AS avg_fat_pct,
       ROUND(AVG(m.snf_pct), 2) AS avg_snf_pct
FROM farmer f
JOIN milk_collection m ON m.farmer_id = f.farmer_id
GROUP BY f.name, m.collection_date
ORDER BY m.collection_date, f.name;
```

name	collection_date	total_litres	avg_fat_pct	avg_snf_pct
Ramesh Kumar	2026-09-20	22.50	4.10	8.50
Suresh Reddy	2026-09-20	8.30	3.80	8.20

Output of Query 10.3

10.4 Top Contributing Farmer

Purpose. Findthefarmerwhohassupplied the most milk overall.

```
SELECT f.name,SUM(m.qty_litres)  AS total_supplied
FROM farmer f
JOIN milk_collection m ON m.farmer_id = f.farmer_id
GROUP BY f.name
ORDER BY total_supplied DESC
LIMIT 1;
```

name	total_supplied
Ramesh Kumar	22.50

Output of Query 10.4

11. UI Design and Screenshots

11.1 Design

The user interface is a single-page website that runs in any browser. The colour scheme is dark green and gold, chosen to suit an agricultural application. The layout has three parts:

- **Top bar** with the application name and a live database status badge that turns green ("MySQL connected") when the server can reach the database, and red otherwise.
- **Tab bar** with seven tabs: Farmers, Cattle, Milk Collection, Rate Chart, Payments, Health Records and Amount Due Report.
- **Work area** for the selected tab, containing an entry form at the top and a data table below. Each row of the table has a Delete button.

Forms use the correct input types (date pickers, numeric fields with decimals, drop-downs). Fields that refer to another table, such as the farmer of a cattle record, are drop-downs filled from the database, so a user cannot enter an owner that does not exist. A short pop-up message confirms every successful insert and delete, and shows the database's error message when an operation fails.

Screen	Operation	What it demonstrates
Farmers tab	View	Rows read from MySQL and displayed in a table
Farmers tab form	Insert	A new farmer is written to the farmer table
Delete button	Delete	A row is removed from the table after confirmation
Cattle / Milk Collection tabs	Insert, View	Foreign-key drop-downs; joined farmer name in the table
Amount Due Report	Query	Presentation-II query run from the UI for a chosen date
Delete of a farmer with child rows	Error handling	Foreign-key RESTRICT stops the deletion

11.2 Screenshots

Figures 11.1to11.7 are screenshots of the running application connected to MySQL. The green badge "MySQL connected" in the top bar confirms the database link in every screen.

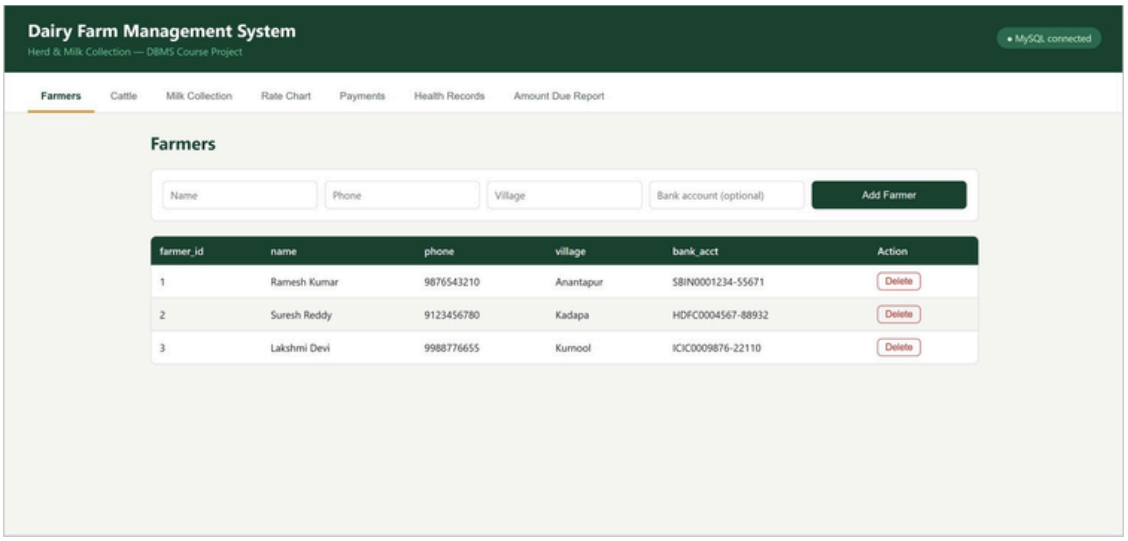


Figure 11.1: Farmers tab — View operation, with the insert form and the farmer table read from MySQL

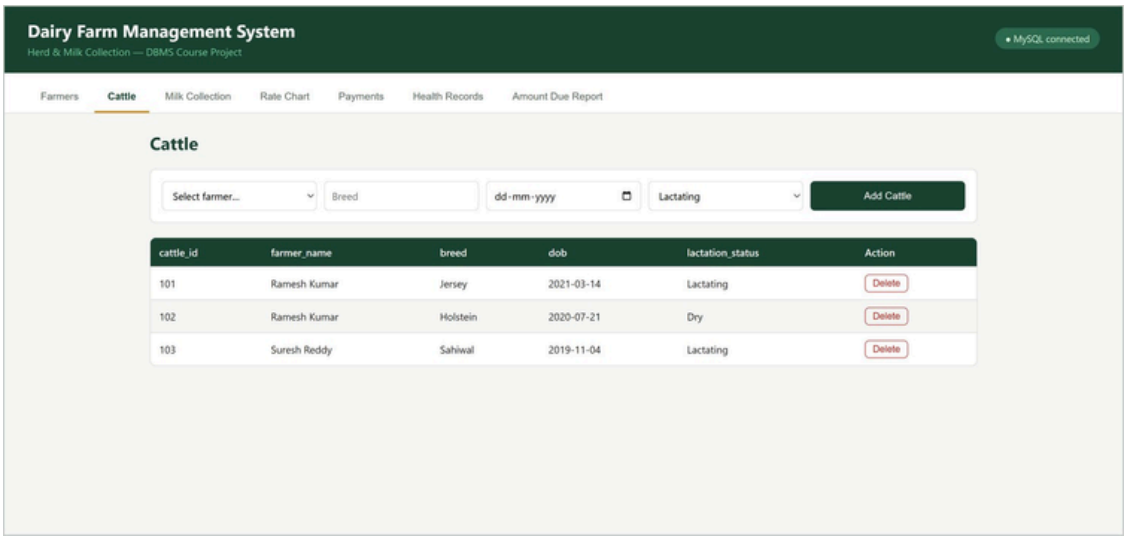


Figure 11.2: Cattle tab — animals listed with the owner's name joined from the farmer table

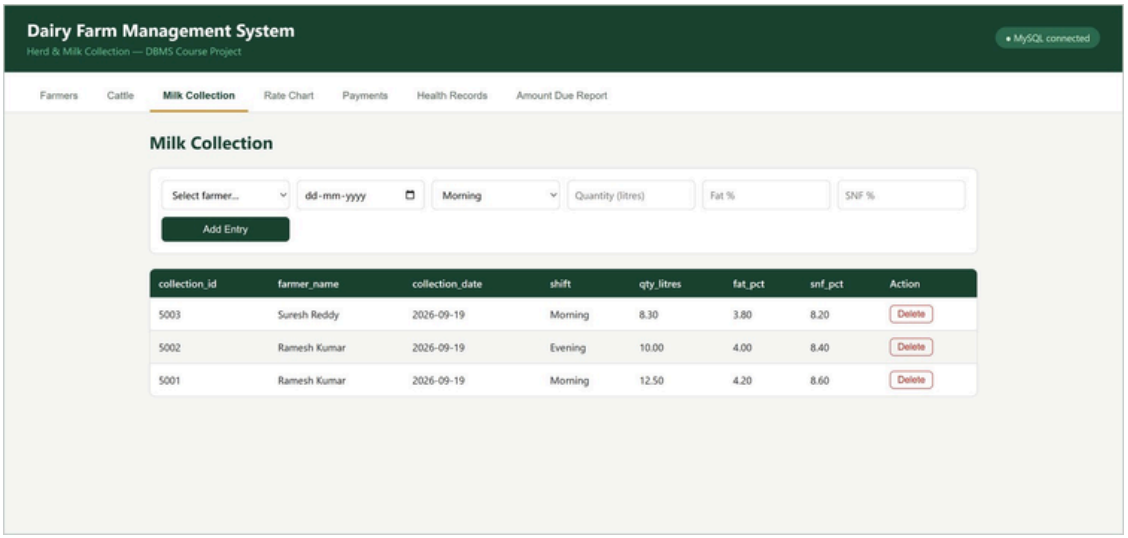


Figure 11.3: Milk Collection tab — entries with shift, quantity, fat % and SNF %

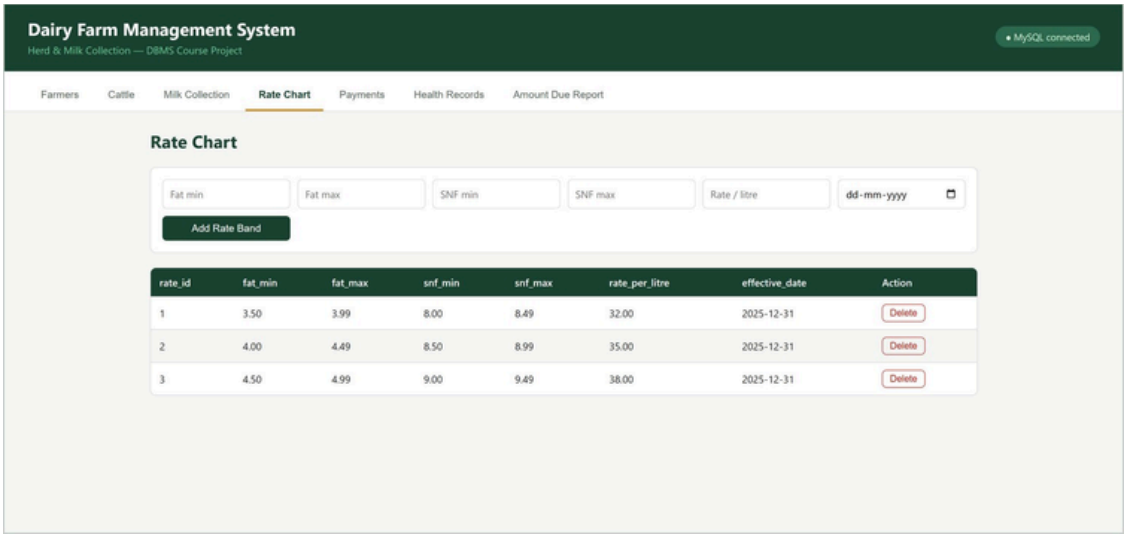


Figure 11.4: Rate Chart tab — non-overlapping fat/SNF price bands

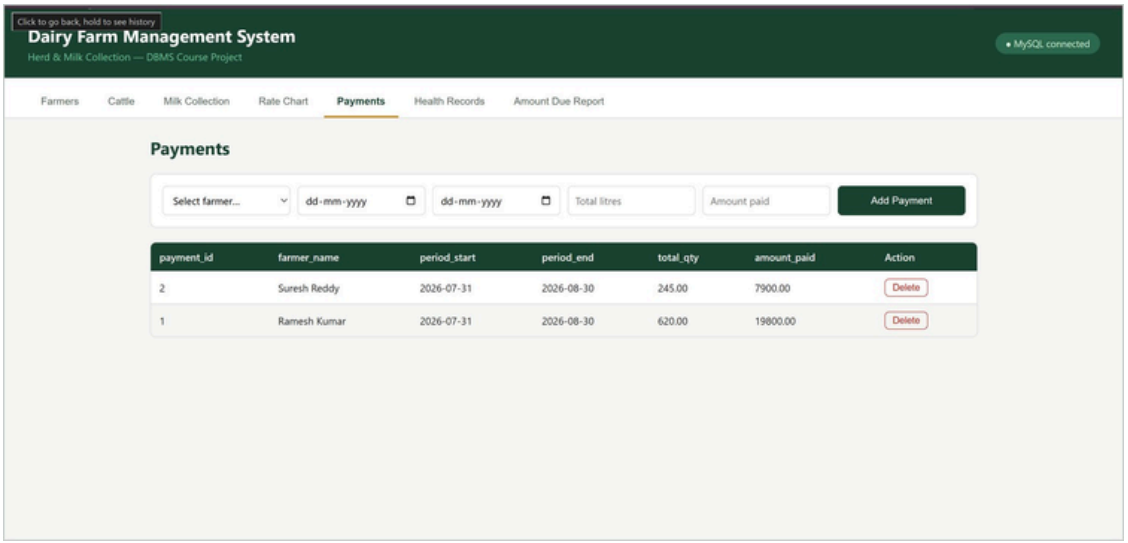


Figure 11.5: Payments tab — payment records per farmer and period

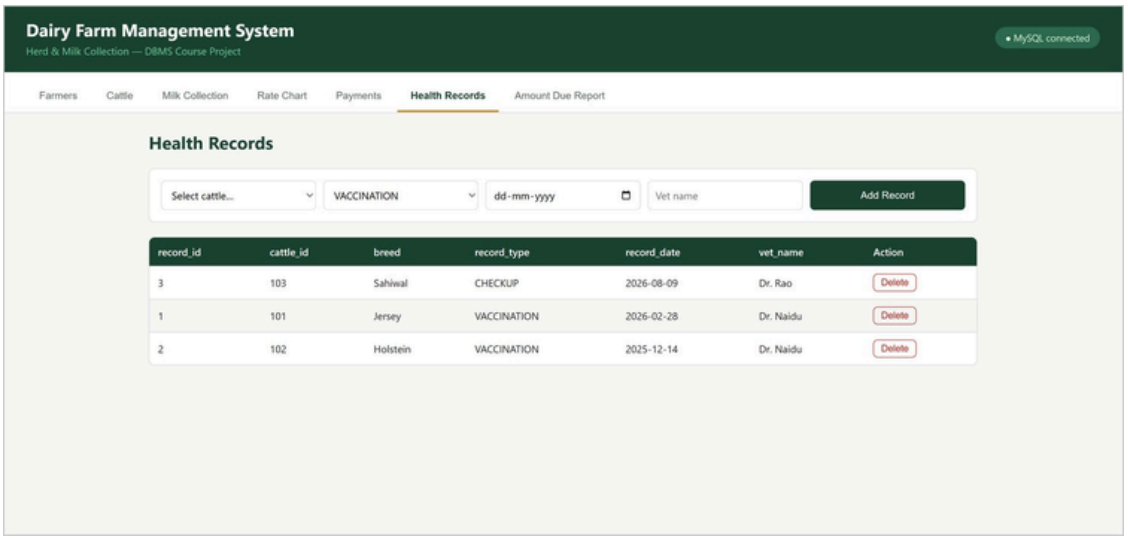


Figure 11.6: Health Records tab — vaccination and checkup history per animal

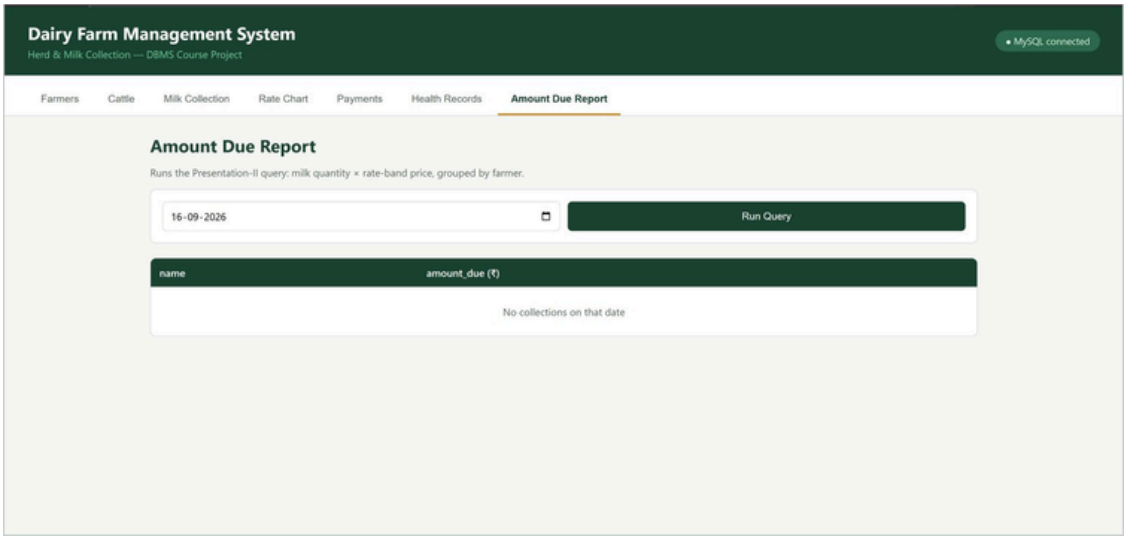


Figure 11.7: Amount Due Report — query run for a date with no collections shows an empty-result message

11.3 Database Tables Behind the UI

The tabs inFigures11.1 to 11.7displaythe rows of the MySQL tables shown below. The screenshots are the output of SELECT * on each table in the MySQL command-line client.

```
mysql> USE dairy_farm_db;
Database changed
mysql>
mysql> SELECT * FROM farmer;
+-----------+--------------+------------+-----------+-------------------+
| farmer_id | name         | phone      | village   | bank_acct         |
+-----------+--------------+------------+-----------+-------------------+
|         1 | Ramesh Kumar | 9876543210 | Anantapur | SBIN0001234-55671 |
|         2 | Suresh Reddy | 9123456780 | Kadapa    | HDFC0004567-88932 |
|         3 | Lakshmi Devi | 9988776655 | Kurnool   | ICIC0009876-22110 |
+-----------+--------------+------------+-----------+-------------------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM cattle;
+-----------+-----------+----------+------------+------------------+
| cattle_id | farmer_id | breed    | dob        | lactation_status |
+-----------+-----------+----------+------------+------------------+
|       101 |         1 | Jersey   | 2021-03-15 | Lactating        |
|       102 |         1 | Holstein | 2020-07-22 | Dry              |
|       103 |         2 | Sahiwal  | 2019-11-05 | Lactating        |
+-----------+-----------+----------+------------+------------------+
3 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM rate_chart;
+---------+---------+---------+---------+---------+----------------+----------------+
| rate_id | fat_min | fat_max | snf_min | snf_max | rate_per_litre | effective_date |
+---------+---------+---------+---------+---------+----------------+----------------+
|       1 |    3.50 |    3.99 |    8.00 |    8.49 |          32.00 | 2026-01-01     |
|       2 |    4.00 |    4.49 |    8.50 |    8.99 |          35.00 | 2026-01-01     |
|       3 |    4.50 |    4.99 |    9.00 |    9.49 |          38.00 | 2026-01-01     |
+---------+---------+---------+---------+---------+----------------+----------------+
3 rows in set (0.009 sec)
```

*Figure 11.8: MySQL output of SELECT * FROM farmer, cattle and rate_chart*

```
mysql>
mysql> SELECT * FROM milk_collection;
+---------------+-----------+-----------------+---------+------------+---------+---------+
| collection_id | farmer_id | collection_date | shift   | qty_litres | fat_pct | snf_pct |
+---------------+-----------+-----------------+---------+------------+---------+---------+
|          5001 |         1 | 2026-09-20      | Morning |      12.50 |    4.20 |    8.60 |
|          5002 |         1 | 2026-09-20      | Evening |      10.00 |    4.00 |    8.40 |
|          5003 |         2 | 2026-09-20      | Morning |       8.30 |    3.80 |    8.20 |
+---------------+-----------+-----------------+---------+------------+---------+---------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM payment;
+------------+-----------+--------------+------------+-----------+-------------+
| payment_id | farmer_id | period_start | period_end | total_qty | amount_paid |
+------------+-----------+--------------+------------+-----------+-------------+
|          1 |         1 | 2026-08-01   | 2026-08-31 |    620.00 |    19800.00 |
|          2 |         2 | 2026-08-01   | 2026-08-31 |    245.00 |     7900.00 |
+------------+-----------+--------------+------------+-----------+-------------+
2 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM health_record;
+-----------+-----------+-------------+-------------+-----------+
| record_id | cattle_id | record_type | record_date | vet_name  |
+-----------+-----------+-------------+-------------+-----------+
|         1 |       101 | VACCINATION | 2026-03-01  | Dr. Naidu |
|         2 |       102 | VACCINATION | 2025-12-15  | Dr. Naidu |
|         3 |       103 | CHECKUP     | 2026-08-10  | Dr. Rao   |
+-----------+-----------+-------------+-------------+-----------+
3 rows in set (0.008 sec)
```

*Figure 11.9: MySQL output of SELECT * FROM milk_collection, payment and health_record*

11.4 Insert, Delete and Report Screenshots

```
mysql> USE dairy_farm_db;
Database changed
mysql>
mysql> SELECT * FROM farmer;
+-----------+--------------+------------+-----------+-------------------+
| farmer_id | name         | phone      | village   | bank_acct         |
+-----------+--------------+------------+-----------+-------------------+
|         1 | Ramesh Kumar | 9876543210 | Anantapur | SBIN0001234-55671 |
|         2 | Suresh Reddy | 9123456780 | Kadapa    | HDFC0004567-88932 |
|         3 | Lakshmi Devi | 9988776655 | Kurnool   | ICIC0009876-22110 |
+-----------+--------------+------------+-----------+-------------------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM cattle;
+-----------+-----------+----------+------------+------------------+
| cattle_id | farmer_id | breed    | dob        | lactation_status |
+-----------+-----------+----------+------------+------------------+
|       101 |         1 | Jersey   | 2021-03-15 | Lactating        |
|       102 |         1 | Holstein | 2020-07-22 | Dry              |
|       103 |         2 | Sahiwal  | 2019-11-05 | Lactating        |
+-----------+-----------+----------+------------+------------------+
3 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM rate_chart;
+---------+---------+---------+---------+---------+----------------+----------------+
| rate_id | fat_min | fat_max | snf_min | snf_max | rate_per_litre | effective_date |
+---------+---------+---------+---------+---------+----------------+----------------+
|       1 |    3.50 |    3.99 |    8.00 |    8.49 |          32.00 | 2026-01-01     |
|       2 |    4.00 |    4.49 |    8.50 |    8.99 |          35.00 | 2026-01-01     |
|       3 |    4.50 |    4.99 |    9.00 |    9.49 |          38.00 | 2026-01-01     |
+---------+---------+---------+---------+---------+----------------+----------------+
3 rows in set (0.009 sec)
```

Figure 11.10: Insert — MySQL output BEFORE adding a record (the farmer table has 3 rows)

```
mysql>
mysql> SELECT * FROM milk_collection;
+---------------+-----------+-----------------+---------+------------+---------+---------+
| collection_id | farmer_id | collection_date | shift   | qty_litres | fat_pct | snf_pct |
+---------------+-----------+-----------------+---------+------------+---------+---------+
|          5001 |         1 | 2026-09-20      | Morning |      12.50 |    4.20 |    8.60 |
|          5002 |         1 | 2026-09-20      | Evening |      10.00 |    4.00 |    8.40 |
|          5003 |         2 | 2026-09-20      | Morning |       8.30 |    3.80 |    8.20 |
+---------------+-----------+-----------------+---------+------------+---------+---------+
3 rows in set (0.008 sec)

mysql>
mysql> SELECT * FROM payment;
+------------+-----------+--------------+------------+-----------+-------------+
| payment_id | farmer_id | period_start | period_end | total_qty | amount_paid |
+------------+-----------+--------------+------------+-----------+-------------+
|          1 |         1 | 2026-08-01   | 2026-08-31 |    620.00 |    19800.00 |
|          2 |         2 | 2026-08-01   | 2026-08-31 |    245.00 |     7900.00 |
+------------+-----------+--------------+------------+-----------+-------------+
2 rows in set (0.009 sec)

mysql>
mysql> SELECT * FROM health_record;
+-----------+-----------+-------------+-------------+-----------+
| record_id | cattle_id | record_type | record_date | vet_name  |
+-----------+-----------+-------------+-------------+-----------+
|         1 |       101 | VACCINATION | 2026-03-01  | Dr. Naidu |
|         2 |       102 | VACCINATION | 2025-12-15  | Dr. Naidu |
|         3 |       103 | CHECKUP     | 2026-08-10  | Dr. Rao   |
+-----------+-----------+-------------+-------------+-----------+
3 rows in set (0.008 sec)
```

12. Implementation Details

12.1 Technology Stack

Layer	Technology	Role
Presentation	HTML5, CSS3, JavaScript	Forms, tables and tabs; calls the API with fetch()
Application	Node.js with Express 4	REST API, serves the website, validates input
Data access	mysql2 (promise pool)	Connects to MySQL; parameterised queries
Database	MySQL 8.0	Stores and enforces the relational schema
Configuration	dotenv	Reads credentials from a .env file

12.2 Architecture

The system follows a three-tier design. The browser never talks to MySQL directly; it sends HTTP requests with JSON to the Express server, which runs SQL through a connection pool and returns the rows as JSON.

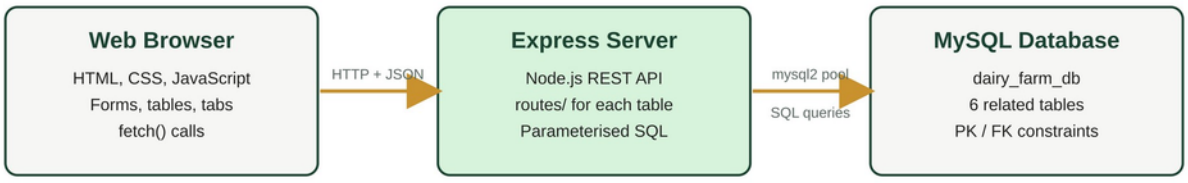


Figure 12.1: Three-tier architecture

12.3 Project Structure

```
dairy-farm-website/
├── backend/
│   ├── server.js            Express app, static files, report route
│   ├── db.js                MySQL connection pool
│   ├── .env.example         Template for DB credentials
│   ├── package.json
│   └── routes/              farmer.js, cattle.js, milkCollection.js,
│                            rateChart.js, payment.js, healthRecord.js
├── frontend/
│   ├── index.html           Tabs, forms and tables
│   ├── style.css
│   └── script.js            View / insert / delete logic
└── database/
    ├── dairy_farm_full.sql  Schema, data and
    └── view_tables.sql      queries SELECT * for
                             every table
```

12.4 Database Connectivity

The connection ismade in `db.js` using a pool of up to ten reusable connections. Host, user, password and database name are read from environment variables defined in `backend/.env`. That file is excluded from version control by `.gitignore`, so credentials are not published; `.env.example` shows the required keys. The option `dateStrings: true` makes MySQL DATE values arrive as plain text, which prevents a timezone conversion from showing dates one day early in the browser.

```
// backend/db.js
require("dotenv").config();
const mysql = require("mysql2/promise");

const pool = mysql.createPool({
  host: process.env.DB_HOST || "localhost",
  user: process.env.DB_USER || "root",
  password: process.env.DB_PASSWORD || "",
  database: process.env.DB_NAME || "dairy_farm_db",
  waitForConnections: true,
  connectionLimit: 10,
  dateStrings: true,      // DATE columns come back as "YYYY-MM-DD" text
});
module.exports = pool;
```

The server checks connectivity at the `/api/health` endpoint, which runs `SELECT 1`. The status badge in the UI calls this endpoint when the page loads.

12.5 REST API

Method	URL	Operation
GET	/api/farmers	View all farmers
POST	/api/farmers	Insert a farmer
DELETE	/api/farmers/:id	Delete a farmer
GET / POST / DELETE	/api/cattle	View, insert, delete cattle (:id for delete)
GET / POST / DELETE	/api/milk-collection /api/rate-	View, insert, delete collection entries
GET / POST / DELETE	chart	View, insert, delete rate bands
GET / POST / DELETE	/api/payments /api/health-	View, insert, delete payments
GET / POST / DELETE	records /api/reports/amount-	View, insert, delete health records
GET	due?date= /api/health	Run the Presentation-II query
GET		Database connectivity check

12.6 Key Code Snippets

Insert a record (backend/routes/farmer.js)

```
router.post("/", async (req, res) => {
  try {
    const { name, phone, village, bank_acct } = req.body;
    if (!name || !phone || !village) {
      return res.status(400).json({ error: "name, phone and village are required" });
    }
    const [result] = await pool.query(
      "INSERT INTO farmer (name, phone, village, bank_acct) VALUES (?, ?, ?, ?)",
      [name, phone, village, bank_acct || null]
    );
    res.status(201).json({ farmer_id: result.insertId, name, phone, village });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
  });
```

Delete a record (backend/routes/farmer.js)

```
router.delete("/:id", async (req,       res)=>{
  try {
    const [result] = await pool.query(
      "DELETE FROM farmer WHERE farmer_id = ?", [req.params.id]);
    if (result.affectedRows === 0)
      return res.status(404).json({ error: "Farmer not found" });
    res.json({ message: "Farmer deleted", farmer_id: req.params.id });
  } catch (err) {
    // foreign key RESTRICT: farmer still has cattle / milk / payments
    res.status(400).json({ error: err.message });
  }
  });
```

All queries use ? placeholders. The values are sent to MySQL separately from the SQL text, which protects against SQL injection.

View records and delete from the UI (frontend/script.js)

```
async function loadTable(key) {
  const { api: url, pk, columns } = ENTITIES[key];
  const rows = await api(url);                          // GET from the REST API
  // build <thead> and <tbody>; every row gets a Delete button
  table.innerHTML = head + rows.map(r => rowHtml(r)).join("");
}

document.addEventListener("click", async (e) => {
  const btn = e.target.closest(".btn-delete");
  if (!btn || !confirm("Delete record " + btn.dataset.id + "?")) return;
  await api(ENTITIES[btn.dataset.key].api + "/" + btn.dataset.id,
            { method: "DELETE" });
  toast("Record deleted");
  loadTable(btn.dataset.key);                           // refresh the table
});
```

Amount due report (backend/server.js)

```
app.get("/api/reports/amount-due", async (req, res) => {
  const date = req.query.date || new Date().toISOString().slice(0, 10);
  const [rows] = await pool.query(
    `SELECT f.name, SUM(m.qty_litres * r.rate_per_litre) AS amount_due
     FROM farmer f
     JOIN milk_collection m ON m.farmer_id = f.farmer_id
     JOIN rate_chart r ON m.fat_pct BETWEEN r.fat_min AND r.fat_max
     WHERE m.collection_date = ?
     GROUP BY f.name`, [date]);
  res.json({ date, rows });
});
```

13. Testing

The tests below cover connectivity, viewing, insertion, deletion, validation, constraint enforcement and the report. Expected results follow from the schema and the sample data. The last column is completed after running each case on the system.

ID	Test case	Steps / input	Expected result	Actualresult/ status
T1	Database connection View farmers View cattle	Start server with correct .env; open the website and /api/health Open Farmers tab	Badge shows "MySQL connected"; /api/health returns status ok 3 sample farmers listed	
T2		Open Cattle tab	3 animals listed with owner names	
T3		Name Test Farmer, phone 9000000001, village Medak → Add Farmer	Success message; row appears; SELECT * FROM farmer shows it	
T4	Insert valid farmer	Leave phone empty → Add Farmer	Form blocks submission; no row added	
T5	Insert with missing field	Insert a farmer using phone 9876543210	MySQL duplicate-entry error shown; no row added	
T6	Duplicate phone	Select farmer 1, breed Gir, status Lactating → Add Cattle	Row added with owner Ramesh Kumar	
T7	Insert cattle	Farmer 1, 2026-09-21, Morning, 9.5 L, fat 4.1, SNF 8.5	Entry added and visible in table and in MySQL	
T8	Insert milk collection	Delete the entry added in T8 and confirm	Row disappears; SELECT confirms it is gone	
T9	Delete record	Delete Ramesh Kumar (has cattle and milk entries)	Deletion blocked by foreign key; error message; row remains	
T10	Delete farmer with dependants			
T11	Delete farmer without dependants	Delete Test Farmer from T4	Row deleted	
T12	Cascade on health records	Delete cattle 101	Cattle row and its health record (id 1) are removed	
T13	Dropdown refresh	Add a farmer, then open the Cattle form	New farmer appears in the farmer drop-down	
T14	Amount due report	Report tab, date 2026-09-20	Ramesh Kumar 787.50; Suresh Reddy 265.60	
T15	Report with no data	Report tab, date 2026-01-01	Message "No collections on that date"	

14. Conclusion and Future Enhancements

14.1 Conclusion

This project produceda working database system for a dairy farm herd and milk collection centre. The

ER model was converted into six tables in Third Normal Form, with primary keys, foreign keys, unique and NOT NULL constraints, and referential actions that protect important history. The amount due to every farmer is computed by a single SQL join against the rate chart, replacing manual lookup. The web interface connected to MySQL lets a user view, insert and delete records in every table, and the database itself rejects invalid operations such as deleting a farmer who still has records.

Working on the project showed how design choices affect correctness. Making the rate bands non-overlapping prevents a boundary value from being counted twice, and storing payments as settled snapshots keeps history stable when rates change.

14.2 Future Enhancements

- User login withroles (collection-centre operator, accountant, administrator).
- Editing of existing records from the UI, with an audit log of changes.
- Automatic payment statement generation using a stored procedure that fills the payment table from collections.
- Direct data capture from electronic milk analysers and weighing scales.
- Breeding, calving and feed-management tables for complete herd management.
- SMS or WhatsApp notification of daily collection and payment to each farmer.
- Dashboards for yield trends, quality trends and vaccination reminders.
- Amobileapplication for field staff and farmers.

15. References

[1] R. Elmasriand S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[3] Oracle Corporation, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[4] OpenJS Foundation, Node.js Documentation. https://nodejs.org/docs/latest/api/

[5] Express.js, Express 4.x API Reference. https://expressjs.com/en/4x/api.html

[6] mysql2 library documentation. https://sidorares.github.io/node-mysql2/docs

[7] MDN Web Docs, Using the Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

16. Appendix: GitHub Repository Link

Repository: https://github.com//DBMS-Course-Project